

Greedy Algorithms

Chapter 5

1

Outline

- Transforming Cabbage into Turnip
- Genome Rearrangements
- Sorting By Reversals
- Greedy Algorithm for Sorting by Reversals
- Pancake Flipping Problem
- Approximation Algorithms
- Breakpoints: a Different Face of Greed

8/29/21

Chapter 5. Greedy Algorithms

2

2

Turnip vs Cabbage: Look and Taste Different

- Although cabbages and turnips share a common ancestor, they look and taste different



8/29/21

Chapter 5. Greedy Algorithms

3

3

Turnip vs Cabbage: Almost Identical mtDNA gene sequences

- In 1980s Jeffrey Palmer studied evolutionary change in plant organelles by comparing mitochondrial genomes of the cabbage and turnip
- 99% similarity between genes
- These surprisingly identical gene sequences differed in gene order
- This helped pave the way to analyzing genome rearrangements in molecular evolution

8/29/21

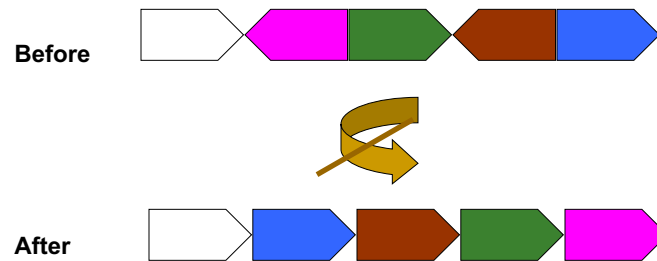
Chapter 5. Greedy Algorithms

4

4

Turnip vs Cabbage: Different mtDNA Gene Order

- Gene order comparison:



Evolution is manifested as the divergence in gene order

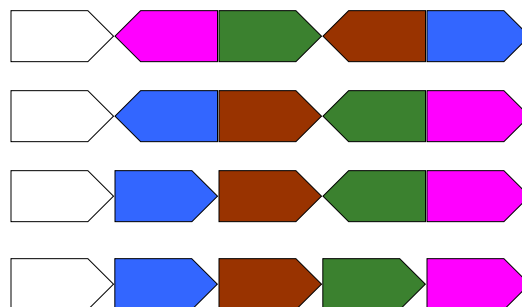
8/29/21

Chapter 5. Greedy Algorithms

5

Turnip vs Cabbage: Different mtDNA Gene Order

- Gene order comparison:

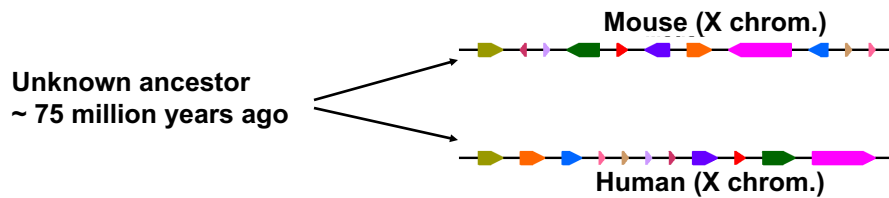


8/29/21

Chapter 5. Greedy Algorithms

6

Genome rearrangements



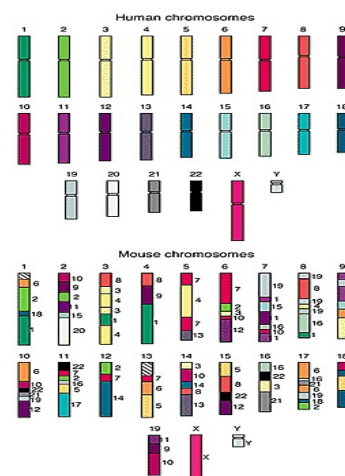
8/29/21

Chapter 5. Greedy Algorithms

7

Mouse vs Human Genome

- Humans and mice have similar genomes, but their genes are ordered differently
- ~245 rearrangements
 - Reversal, fusion, fission, translocation
- **Reversal**: flipping a block of genes within a genomic sequence



8/29/21

Chapter 5. Greedy Algorithms

8

Waardenburg's syndrome

- Genetic disorder
- Characterized by loss of hearing and pigmentary dysphasia
- Gene implicated in the disease on human was linked to chromosome 2 but it was not clear where exactly it is located on chromosome 2



8/29/21

Chapter 5. Greedy Algorithms

9

9

Waardenburg's syndrome and splotch mice

- A breed of mice (with splotch gene) had similar symptoms caused by the same type of gene as in humans
- Finding the gene in mice gives clues to where the same gene is located in humans
- Scientists succeeded in identifying location of gene responsible for disorder in mice

8/29/21

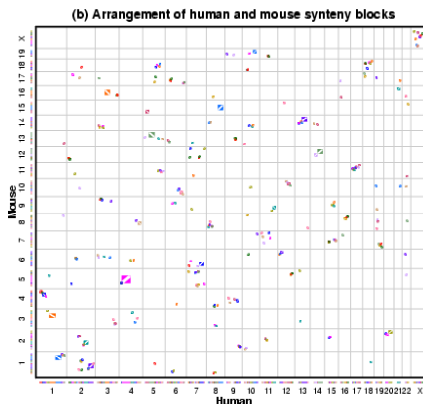
Chapter 5. Greedy Algorithms

10

10

Comparative Genomic Architecture of Human and Mouse Genomes

- To locate where corresponding gene is in humans, we have to analyze the relative architecture of genes of humans and mouse



8/29/21

Chapter 5. Greedy Algorithms

11

11

Reversals

- Let's first assume that genes in genomic segment π does not have direction

$$\pi = \pi_1 \text{ ----- } \pi_{i-1} \pi_i \pi_{i+1} \text{ ----- } \pi_{j-1} \pi_j \pi_{j+1} \text{ ----- } \pi_n$$

$$\rho(i, j)$$

$$\pi' = \pi_1 \text{ ----- } \pi_{i-1} \pi_j \pi_{j-1} \text{ ----- } \pi_{i+1} \pi_i \pi_{j+1} \text{ ----- } \pi_n$$

- $\rho(i, j)$ is defined as the operation that reverses the gene sequence of π (original genomic segment) between i and j to π'

8/29/21

Chapter 5. Greedy Algorithms

12

12

Reversals: Example

- Example:

$$\pi = 1 \ 2 \ 4 \ 3 \ 7 \ 5 \ 6$$

$\rho(3,6)$

$$\pi' = 1 \ 2 \ 5 \ 7 \ 3 \ 4 \ 6$$

8/29/21

Chapter 5. Greedy Algorithms

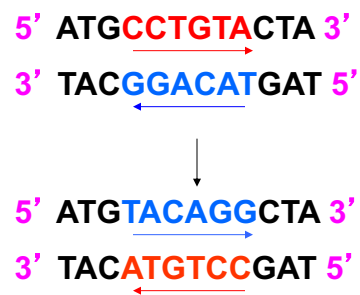
13

13

Reversals: Example



Break
and
Invert



8/29/21

Chapter 5. Greedy Algorithms

14

14

Reversal Distance Problem

- Goal: Given two permutations, find the shortest series of reversals that forms one from the other
- Input: Permutations π and σ
- Output: A series of reversals $\rho_1, \dots, \rho_t$ transforming π into σ , such that t is minimum
- t = reversal distance between π and σ
- $d(\pi, \sigma)$ denotes smallest possible value of t , given π and σ

8/29/21

Chapter 5. Greedy Algorithms

15

15

Sorting By Reversals Problem

- Goal: Given a permutation, find a shortest series of reversals that transforms it into the identity permutation $(1\ 2\ \dots\ n)$
- Input: permutation π
- Output: A series of reversals $\rho_1, \dots, \rho_t$ transforming π into the identity permutation such that t is minimum

8/29/21

Chapter 5. Greedy Algorithms

16

16

Sorting By Reversals: Example

- Minimal value of t is denoted as $d(\pi)$ and this value is the reversal distance of permutation π

- Example :

input: $\pi = 3 \ 4 \ 2 \ 1 \ 5 \ 6 \ 7 \ 10 \ 9 \ 8$
 output: $\begin{array}{cccccccccc} 4 & 3 & 2 & 1 & 5 & 6 & 7 & 10 & 9 & 8 \\ 4 & 3 & 2 & 1 & 5 & 6 & 7 & 8 & 9 & 10 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \end{array}$

So $t = 3$

8/29/21

Chapter 5. Greedy Algorithms

17

17

Sorting By Reversals: A Greedy Algorithm

- If sorting permutation $\pi = 1 \ 2 \ 3 \ 6 \ 4 \ 5$, the first three numbers are already in order so it does not make any sense to break them. These already sorted numbers of π will be defined as $prefix(\pi)$
 - $prefix(\pi) = 3$
- This results in an idea for an greedy algorithm to sort by reversals; increase $prefix(\pi)$ at every step

8/29/21

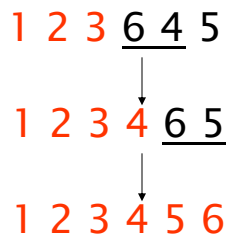
Chapter 5. Greedy Algorithms

18

18

Greedy Algorithm: An Example

- Doing so, π can be sorted



- $d(\pi) = 2$
- Number of steps to sort permutation of length n is at most $(n - 1)$

8/29/21

Chapter 5. Greedy Algorithms

19

19

Greedy Algorithm: Pseudocode

SimpleReversalSort(π)

```
1 for  $i \leftarrow 1$  to  $n - 1$ 
2    $j \leftarrow$  position of element  $i$  in  $\pi$  (i.e.,  $\pi_j = i$ )
3   if  $j \neq i$ 
4      $\pi \leftarrow \pi * \rho(i, j)$ 
5   output  $\pi$ 
6 if  $\pi$  is the identity permutation
7   return
```

8/29/21

Chapter 5. Greedy Algorithms

20

20

Analyzing SimpleReversalSort

- **Greedy** algorithm; does not guarantee the smallest number of reversals
- For example, let $\pi = 6\ 1\ 2\ 3\ 4\ 5\dots$
- *SimpleReversalSort*(π) takes five steps:
 - Step 1: 1 **6** **2** 3 4 5
 - Step 2: 1 2 **6** **3** 4 5
 - Step 3: 1 2 3 **6** **4** 5
 - Step 4: 1 2 3 4 **6** **5**
 - Step 5: 1 2 3 4 5 6

8/29/21

Chapter 5. Greedy Algorithms

21

21

Analyzing SimpleReversalSort (cont' d)

- But it can be done in two steps:
$$\pi = 6\ 1\ 2\ 3\ 4\ 5$$
 - Step 1: **5** **4** **3** **2** **1** 6
 - Step 2: 1 2 3 4 5 6
- So, *SimpleReversalSort*(π) is a terrible algorithm that is not optimal

8/29/21

Chapter 5. Greedy Algorithms

22

22

Pancake Flipping Problem

Problem Description

- The chef is sloppy; he prepares an unordered stack of pancakes of different sizes
- The waiter wants to rearrange them (so that the smallest winds up on top, and so on, down to the largest at the bottom)
- He does it by flipping over several from the top, repeating this as many times as necessary

8/29/21

Chapter 5. Greedy Algorithms

23

23

Pancake Flipping Problem: Formulation

- Goal: Given n pancakes, what is the minimum number of flips required to rearrange them?
- Input: An ordered stack of n pancakes of distinct size $1 < 2 < 3 < \dots < n$
- Output: An ordered stack of pancakes, smallest on top, largest on the bottom

8/29/21

Chapter 5. Greedy Algorithms

24

24

Pancake Flipping Problem: Greedy Algorithm

- Greedy approach: 2 prefix reversals at most to place a pancake in its right position, $2n - 2$ steps total at most
- William Gates and Christos Papadimitriou showed in the mid-1970s that this problem can be solved by at most $5/3 (n + 1)$ *prefix reversals*
- Approximation algorithms are useful...

8/29/21

Chapter 5. Greedy Algorithms

25

25

Approximation Algorithms

- Optimal algorithms are unknown for many problems; approximation algorithms are used
- These algorithms find approximate solutions rather than optimal solutions
- The approximation ratio of an algorithm A on input π is:
 - $A(\pi) / \text{OPT}(\pi)$
 - $A(\pi)$ is the solution produced by algorithm A and $\text{OPT}(\pi)$ is the optimal solution of the problem

8/29/21

Chapter 5. Greedy Algorithms

26

26

Approximation Ratio/Performance Guarantee

- Approximation ratio (performance guarantee) of algorithm A: max approximation ratio of all inputs of size n
- For algorithm A that minimizes objective function (minimization algorithm):
 - $\max_{|\pi| = n} A(\pi) / \text{OPT}(\pi)$
- For maximization algorithm:
 - $\min_{|\pi| = n} A(\pi) / \text{OPT}(\pi)$

8/29/21

Chapter 5. Greedy Algorithms

27

27

Breakpoints

$$\pi = \pi_1 \pi_2 \pi_3 \dots \pi_{n-1} \pi_n$$

- A pair of elements π_i and π_{i+1} are adjacent if

$$\pi_{i+1} = \pi_i \pm 1$$

- For example:

$$\pi = 1 \ 9 \ \underline{3} \ \underline{4} \ \underline{7} \ \underline{8} \ 2 \ \underline{6} \ 5$$

- (3, 4) or (7, 8) and (6,5) are adjacent pairs

8/29/21

Chapter 5. Greedy Algorithms

28

28

Breakpoints: An Example

There is a breakpoint between any pair of non-adjacent elements:

$$\pi = 1 \mid 9 \mid 3 \mid 4 \mid 7 \mid 8 \mid 2 \mid 6 \mid 5$$

- Pairs $(1,9)$, $(9,3)$, $(4,7)$, $(8,2)$ and $(2,6)$ form breakpoints of permutation π

Extending Permutations

- We put two blocks π_0 and π_{n+1} at the ends of π
 $\pi_0 = 0$ and $\pi_{n+1} = n + 1$
- This gives us the goal to sort the elements between the end blocks to the identity permutation
- Example: $\pi = 1 \mid 9 \mid 3 \mid 4 \mid 7 \mid 8 \mid 2 \mid 6 \mid 5$

↓ Extending with 0 and 10

$$\pi = 0 \mid 1 \mid 9 \mid 3 \mid 4 \mid 7 \mid 8 \mid 2 \mid 6 \mid 5 \mid 10$$

Note: A new breakpoint was created after extending

Reversal Distance and Breakpoints

- $b(\pi)$ = number of breakpoints
- Each reversal eliminates at most 2 breakpoints. This implies that $d(\pi) \geq b(\pi) / 2$

$\pi = 2 \ 3 \ 1 \ 4 \ 6 \ 5$

0 | 2 3 | 1 | 4 | 6 | 5 | 7

$b(\pi) = 5$

0 | 1 | 3 2 | 4 | 6 | 5 | 7

$b(\pi) = 4$

0 | 1 | 2 | 3 | 4 | 6 5 | 7

$b(\pi) = 2$

0 | 1 | 2 | 3 | 4 | 5 | 6 | 7

$b(\pi) = 0$

8/29/21

Chapter 5. Greedy Algorithms

31

31

Sorting By Reversals: A Better Greedy Algorithm

BreakPointReversalSort(π)

- 1 while $b(\pi) > 0$
- 2 Among all possible reversals,
 choose ρ minimizing $b(\pi * \rho)$
- 3 $\pi \leftarrow \pi * \rho(i, j)$
- 4 output π
- 5 return

8/29/21

Chapter 5. Greedy Algorithms

32

32

Things To Consider

- Is BreakPointReversalSort a better algorithm than SimpleReversalSort ?
- Does BreakPointReversalSort algorithm terminate?
 - How can we be sure that removing some breakpoints does not introduce others, leading to an endless cycle.

8/29/21

Chapter 5. Greedy Algorithms

33

33

Strips

- Strip: an interval between two consecutive breakpoints in π
- Decreasing strips: *strips* that are in decreasing order (e.g. 6 5 and 3 2). 
- Increasing strips: *strips* that are in increasing order (e.g. 7 8). 
- A single-element strip can be increasing or decreasing.  

8/29/21

Chapter 5. Greedy Algorithms

34

34

Strips: An Example

- For permutation 1 9 3 4 7 8 2 5 6:
 - There are 7 strips:

0 1 9 4 3 7 8 2 5 6 10

→ ← ← → ← → ←

8/29/21

Chapter 5. Greedy Algorithms

35

35

Things To Consider

- Fact 1:
 - If permutation π contains at least one decreasing strip, then there exists a reversal ρ which decreases the number of breakpoints (i.e. $b(\pi * \rho) < b(\pi)$)

8/29/21

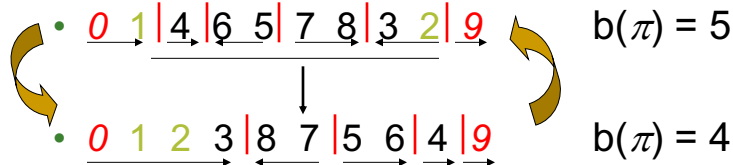
Chapter 5. Greedy Algorithms

36

36

Things To Consider (cont' d)

- For $\pi = 1\ 4\ 6\ 5\ 7\ 8\ 3\ 2$
- $0\ 1\ 4\ 6\ 5\ 7\ 8\ 3\ 2\ 9$ $b(\pi) = 5$
 - Choose decreasing strip with the smallest element k in π ($\pi_8 = k=2, j=8$)
 - Find $k - 1$ in the permutation ($\pi_1 = k-1=1, i=1$), then reverse if $i < j$ $\rho(i+1, j); \rho(j+1, i)$



8/29/21

Chapter 5. Greedy Algorithms

37

37

Things To Consider (cont' d)

- Fact 2:
 - If there is no decreasing strip, a reversal ρ might reduce or might not reduce the number of breakpoints.
 - By reversing an increasing strip (# of breakpoints stay unchanged), the number of strips can be reduced in the next step.

8/29/21

Chapter 5. Greedy Algorithms

38

38

Things To Consider (cont' d)

- There are no decreasing strips in π , for:

$$\begin{array}{lcl} \pi = \underline{0} \ 1 \ 2 \mid 5 \ 6 \ 7 \mid 3 \ 4 \mid \underline{8} & & b(\pi) = 3 \\ \pi * \rho(6,7) = \underline{0} \ 1 \ 2 \mid 5 \ 6 \ 7 \mid 4 \ 3 \mid \underline{8} & & b(\pi) = 3 \end{array}$$

- ✓ $\rho(6,7)$ does not change the # of breakpoints
- ✓ $\rho(6,7)$ creates a decreasing strip, guaranteeing that the next step will decrease the # of breakpoints.

8/29/21

Chapter 5. Greedy Algorithms

39

39

ImprovedBreakpointReversalSorting

ImprovedBreakpointReversalSort(π)

```

1 while  $b(\pi) > 0$ 
2   if  $\pi$  has a decreasing strip
3     Among all possible reversals, choose  $\rho$  that
       minimizes  $b(\pi * \rho)$ 
4   else
5     Choose a reversal  $\rho$  that flips an increasing strip in  $\pi$ 
6    $\pi \leftarrow \pi * \rho$ 
7   output  $\pi$ 
8   return
    
```

8/29/21

Chapter 5. Greedy Algorithms

40

40

ImprovedBreakpointReversalSorting: Performance Guarantee

- *ImprovedBreakPointReversalSort* is an approximation algorithm with a performance guarantee of at most 4
 - It eliminates at least one breakpoint in every two steps; at most $2b(\pi)$ steps $\Rightarrow A(\pi) \leq 2b(\pi)$
 - Optimal algorithm eliminates 2 breakpoints in every step: $d(\pi) \geq b(\pi) / 2$
 - Approximation ratio: $A(\pi) / d(\pi)$
 - Performance guarantee:
 - $(2b(\pi) / d(\pi)) \leq \lceil 2b(\pi) / (b(\pi) / 2) \rceil = 4$

8/29/21

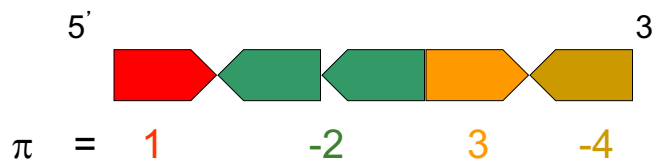
Chapter 5. Greedy Algorithms

41

41

Signed Permutations

- Up to this point, all permutations to sort were unsigned
- But genes have directions... so we should consider signed permutations



8/29/21

Chapter 5. Greedy Algorithms

42

42

GRIMM Web Server

- Real genome architectures are represented by signed permutations
- Efficient algorithms to sort signed permutations have been developed
- GRIMM web server computes the reversal distances between signed permutations:

<http://nbcrc.sdsc.edu/GRIMM/grimm.cgi>

8/29/21

Chapter 5. Greedy Algorithms

43

43

A greedy approach to Motif finding

- Gary Stormo and Gerald Hertz developed CONCENUS.
- Find the two closest l -mers using sequences 1 and 2, $O(l n^2)$
- At each of the remaining $t-2$ iterations, scanning the i -th sequence and select the l -mer that has the maximum $\text{Score}(\mathbf{s}, i)$
- $O(l n^2 + l n t) \ll O(l n^t)$ *SimpleMotifSearch* $\ll O(4^l n t)$ *BruteForceMedianString*

8/29/21

Chapter 5. Greedy Algorithms

44

44

Greedy Motif Search

```
GREEDYMOTIFSEARCH( $DNA, t, n, l$ )
1  bestMotif  $\leftarrow (1, 1, \dots, 1)$ 
2   $s \leftarrow (1, 1, \dots, 1)$ 
3  for  $s_1 \leftarrow 1$  to  $n - l + 1$ 
4    for  $s_2 \leftarrow 1$  to  $n - l + 1$ 
5      if  $Score(s, 2, DNA) > Score(\text{bestMotif}, 2, DNA)$ 
6         $BestMotif_1 \leftarrow s_1$ 
7         $BestMotif_2 \leftarrow s_2$ 
8   $s_1 \leftarrow BestMotif_1$ 
9   $s_2 \leftarrow BestMotif_2$ 
10 for  $i \leftarrow 3$  to  $t$ 
11   for  $s_i \leftarrow 1$  to  $n - l + 1$ 
12     if  $Score(s, i, DNA) > Score(\text{bestMotif}, i, DNA)$ 
13        $bestMotif_i \leftarrow s_i$ 
14    $s_i \leftarrow bestMotif_i$ 
15 return bestMotif
```